

LUT_Generation using Pyhon and ngspice

Testbench for LUT generation

Ngspice netlist

```
* NMOS and PMOS LUT Characterization Testbench

* 1. Include the Technology Library File (Using Absolute Path)
.include "C:\Users\Mohamed\Desktop\ML\LTSPICE_Work\gf180_spice_models_20240101\cmos_tt.txt"

* 2. Operating Parameters
.param Ld=1e-06
.param Wd=1e-05
.param VX=1.0
.param VY=1.5

* 3. Voltage Sources
VDSN Vdn 0 DC {VY}
VGSN vgn 0 DC {VX}
VDSP 0 Vdp DC {VY}
VGSP 0 Vgp DC {VX}

* 4. Transistor Instances
M1 vdn vgn 0 0 nmos_3p3 l={Ld} w={Wd}
M2 vdp vgp 0 0 pmos_3p3 l={Ld} w={Wd}

* 5. Simulation Directives
.op

* 6. Simulation Control Block
.control
    * Pre-instruct Ngspice to save the operating point vector data
    save @m1[vgs] @m1[id] @m1[gm] @m1[gds] @m1[gmb] @m1[vth] @m1[cgs] @m1[cgb] @m1[cgd]
    + @m2[vgs] @m2[id] @m2[gm] @m2[gds] @m2[gmb] @m2[vth] @m2[cgs] @m2[cgb] @m2[cgd]

    run

    set filetype=ascii

    * Write all saved vectors out to the raw file
    write TEST_NETLIST_1.raw

    quit
.endc
```

```
.end
```

Test ngspice calling and reading output

```
import subprocess
import os

def run_simulation(working_dir, netlist_name, raw_data_name, params=None):
    """
    Runs Ngspice and returns a dictionary of extracted parameters.
    params: dict of parameters to update in the netlist (e.g., {'Ld': '2e-6'})
    """
    netlist_path = os.path.join(working_dir, netlist_name)
    raw_path = os.path.join(working_dir, raw_data_name)

    # 1. (Optional) Update Netlist Params here if needed
    # For now, we assume the netlist is already set up correctly.

    # 2. Run Ngspice
    result = subprocess.run(
        [r"C:\Spice64\bin\ngspice_con.exe", "-b", netlist_name],
        cwd=working_dir,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True
    )

    # 3. Parse Output
    if not os.path.exists(raw_path):
        print("Error: Ngspice did not generate the output file.")
        print(result.stderr)
        return None

    extracted_data = {}
    variable_names = {}

    with open(raw_path, 'r') as f:
        lines = f.readlines()

    reading_vars = False
    reading_vals = False
    current_var_index = 0

    for line in lines:
        line = line.strip()
        if not line: continue
```

```

if line.startswith("Variables:"):
    reading_vars = True
    reading_vals = False
    continue
elif line.startswith("Values:"):
    reading_vars = False
    reading_vals = True
    current_var_index = 0
    continue

if reading_vars:
    parts = line.split()
    if len(parts) >= 2:
        variable_names[int(parts[0])] = parts[1]

elif reading_vals:
    parts = line.split()
    if len(parts) == 2:
        current_var_index = 0
        var_value = float(parts[1])
    elif len(parts) == 1:
        current_var_index += 1
        var_value = float(parts[0])
    else: continue

    if current_var_index in variable_names:
        extracted_data[variable_names[current_var_index]] = var_value

return extracted_data

```

```

# --- Execution ---

working_dir = r"C:\Users\Mohamed\Desktop\ML\CAD_projects\Netlists\GF_180nm_tech_work\LUT_gen"
data = run_simulation(working_dir, "TEST_NETLIST_1.sp", "TEST_NETLIST_1.raw")

if data:
    # Print only the parameters you want to see
    print("Simulation complete! Results:")
    for name, value in data.items():
        print(f"{name: <20} : {value:.6e}")

```

```

Simulation complete! Results:
v(@m1[vgs])      : 1.0000000e+00
@m1[cgb]         : -3.870955e-15
@m1[cgd]         : -1.718050e-18
@m1[cgs]         : -2.512119e-14
@m1[gds]         : 1.345650e-06
@m1[gm]          : 4.330858e-04

```

```

@m1[gmbs]      : 1.648896e-04
i(@m1[id])     : 7.790858e-05
v(@m1[vth])    : 6.884356e-01
@m2[cgb]       : -5.731636e-15
@m2[cgd]       : -1.845623e-18
@m2[cgs]       : -2.630786e-14
@m2[gds]       : 1.378490e-07
@m2[gm]        : 7.001797e-05
@m2[gmbs]      : 3.303154e-05
i(@m2[id])     : 8.750635e-06
v(@m2[vgs])    : 1.000000e+00
v(@m2[vth])    : 7.839372e-01

```

LUT Tensor Builder

```

import subprocess
import os
import re
import numpy as np

class LUTTensorBuilder:
    def __init__(self, working_dir, netlist_name="LUT_GENERATOR.sp",
raw_name="LUT_GENERATOR.raw"):
        self.working_dir = working_dir
        self.netlist_path = os.path.join(working_dir, netlist_name)
        self.raw_path = os.path.join(working_dir, raw_name)
        self.ngspice_exe = r"C:\Spice64\bin\ngspice_con.exe"

    def _update_length(self, l_val):
        """Python modifies the netlist to inject the new L for this step."""
        with open(self.netlist_path, 'r') as f:
            content = f.read()

        updated_content = re.sub(r"\.param\s+Ld=\S+", f".param Ld={l_val}", content,
flags=re.IGNORECASE)

        with open(self.netlist_path, 'w') as f:
            f.write(updated_content)

    def _run_sim_and_parse(self):
        """Runs the Ngspice VX sweep and extracts the 1D arrays."""
        subprocess.run(
            [self.ngspice_exe, "-b", os.path.basename(self.netlist_path)],
            cwd=self.working_dir,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            text=True
        )

```

```

variable_names = {}
extracted_data = {}

with open(self.raw_path, 'r') as f:
    lines = f.readlines()

reading_vars = False
reading_vals = False
current_var_index = 0

for line in lines:
    line = line.strip()
    if not line: continue

    if line.startswith("Variables:"):
        reading_vars = True
        continue
    elif line.startswith("Values:"):
        reading_vars = False
        reading_vals = True
        for idx in variable_names:
            extracted_data[variable_names[idx]] = []
        continue

    if reading_vars:
        parts = line.split()
        if len(parts) >= 2:
            var_name = parts[1]
            if var_name.startswith("v(") or var_name.startswith("i("):
                if var_name.endswith(")"):
                    var_name = var_name[2:-1]
                variable_names[int(parts[0])] = var_name

    elif reading_vals:
        parts = line.split()
        if len(parts) == 2:
            current_var_index = 0
            val = float(parts[1])
        elif len(parts) == 1:
            current_var_index += 1
            val = float(parts[0])
        else: continue

        if current_var_index in variable_names:
            extracted_data[variable_names[current_var_index]].append(val)

for k in extracted_data:
    extracted_data[k] = np.array(extracted_data[k])

```

```

        return extracted_data

def generate_tensors(self, l_start, l_stop, l_step):
    """THIS IS THE L SWEEP: It builds the N x M matrix across all lengths."""
    L_vec = np.arange(l_start, l_stop + (l_step/2), l_step)
    N = len(L_vec)

    tensors = {}
    VX_vec = None

    print(f"Initializing matrix generation. Total Length steps (N) = {N}")

    # --- THE L SWEEP LOOP ---
    for i, L in enumerate(L_vec):
        # 1. Update netlist with current L
        self._update_length(f"{L:.6e}")

        # 2. Run simulation and grab the M-point voltage vectors
        data_1d = self._run_sim_and_parse()

        # 3. Dynamically allocate N x M matrices on the very first loop
        if i == 0:
            VX_vec = data_1d['v-sweep']
            M = len(VX_vec)
            print(f"Detected Voltage points (M) = {M}. Matrix shape will be {N}x{M}.")

            for key in data_1d.keys():
                if key != 'v-sweep':
                    tensors[key] = np.zeros((N, M))

        # 4. Slot the 1D arrays into the i-th row of our matrices
        for key in data_1d.keys():
            if key != 'v-sweep':
                tensors[key][i, :] = data_1d[key]

        if i % 50 == 0 or i == N - 1:
            print(f"Processed row {i+1}/{N} -> L = {L*1e6:.3f} um")

    return L_vec, VX_vec, tensors

```

```

w_dir = r"C:\Users\Mohamed\Desktop\ML\CAD_projects\Netlists\GF_180nm_tech_work\LUT_gen"

builder = LUTTensorBuilder(working_dir=w_dir)

# Executes the sweep: 0.3 μm to 20 μm with 10 nm (0.01 μm) step
L_vector, VX_vector, LUT_Matrices = builder.generate_tensors(
    l_start=0.3e-6,

```

```

l_stop=20e-6,
l_step=0.01e-6
)

print("\nGeneration Complete! Validating Tensor Data:\n")
print(f"L Vector shape (N): {L_vector.shape}")
print(f"VX Vector shape (M): {VX_vector.shape}")

print(f"NMOS ID Matrix shape (NxM): {LUT_Matrices['@m1[id]'].shape}")
print(f"PMOS GM Matrix shape (NxM): {LUT_Matrices['@m2[gm]'].shape}")

```

```

Initializing matrix generation. Total Length steps (N) = 1971
Detected Voltage points (M) = 301. Matrix shape will be 1971x301.
Processed row 1/1971 -> L = 0.300 um
Processed row 51/1971 -> L = 0.800 um
Processed row 101/1971 -> L = 1.300 um
Processed row 151/1971 -> L = 1.800 um
Processed row 201/1971 -> L = 2.300 um
Processed row 251/1971 -> L = 2.800 um
Processed row 301/1971 -> L = 3.300 um
Processed row 351/1971 -> L = 3.800 um
Processed row 401/1971 -> L = 4.300 um
Processed row 451/1971 -> L = 4.800 um
Processed row 501/1971 -> L = 5.300 um
Processed row 551/1971 -> L = 5.800 um
Processed row 601/1971 -> L = 6.300 um
Processed row 651/1971 -> L = 6.800 um
Processed row 701/1971 -> L = 7.300 um
Processed row 751/1971 -> L = 7.800 um
Processed row 801/1971 -> L = 8.300 um
Processed row 851/1971 -> L = 8.800 um
Processed row 901/1971 -> L = 9.300 um
Processed row 951/1971 -> L = 9.800 um
Processed row 1001/1971 -> L = 10.300 um
Processed row 1051/1971 -> L = 10.800 um
Processed row 1101/1971 -> L = 11.300 um
Processed row 1151/1971 -> L = 11.800 um
Processed row 1201/1971 -> L = 12.300 um
Processed row 1251/1971 -> L = 12.800 um
Processed row 1301/1971 -> L = 13.300 um
Processed row 1351/1971 -> L = 13.800 um
Processed row 1401/1971 -> L = 14.300 um
Processed row 1451/1971 -> L = 14.800 um
Processed row 1501/1971 -> L = 15.300 um
Processed row 1551/1971 -> L = 15.800 um
Processed row 1601/1971 -> L = 16.300 um
Processed row 1651/1971 -> L = 16.800 um
Processed row 1701/1971 -> L = 17.300 um

```

```
Processed row 1751/1971 -> L = 17.800 um
Processed row 1801/1971 -> L = 18.300 um
Processed row 1851/1971 -> L = 18.800 um
Processed row 1901/1971 -> L = 19.300 um
Processed row 1951/1971 -> L = 19.800 um
Processed row 1971/1971 -> L = 20.000 um
```

Generation Complete! Validating Tensor Data:

```
L Vector shape (N): (1971,)
VX Vector shape (M): (301,)
NMOS ID Matrix shape (NxM): (1971, 301)
PMOS GM Matrix shape (NxM): (1971, 301)
```

Plotting Using the Tensor Data

```
import matplotlib.pyplot as plt
import numpy as np

# 1. Select target lengths to plot (in meters)
target_lengths = [0.3e-6, 0.5e-6, 1.0e-6, 2.0e-6, 5.0e-6, 10.0e-6, 20.0e-6]

# Find the closest actual indices in L_vector for our target lengths
plot_indices = []
for target in target_lengths:
    idx = (np.abs(L_vector - target)).argmin()
    plot_indices.append(idx)

# Extract NMOS data vectors
gm_matrix = LUT_Matrices['@m1[gm]']
id_matrix = LUT_Matrices['@m1[id]']
gds_matrix = LUT_Matrices['@m1[gds]']

# Avoid division by zero in weak inversion / cutoff
# Mask out currents below 10 pA to avoid noisy asymptotes at VGS = 0V
id_masked = np.where(id_matrix > 1e-11, id_matrix, np.nan)
gds_masked = np.where(gds_matrix > 1e-12, gds_matrix, np.nan)

# Compute parameters
gm_over_id = gm_matrix / id_masked
intrinsic_gain = gm_matrix / gds_masked

# 2. Plotting Setup (Side-by-Side Professional Layout)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

for idx in plot_indices:
    L_val_um = L_vector[idx] * 1e6
    label_text = f"L = {L_val_um:.2f} um"
```



```

# Plot 1: gm/ID vs VGS
ax1.plot(VX_vector, gm_over_id[idx, :], label=label_text, linewidth=2)

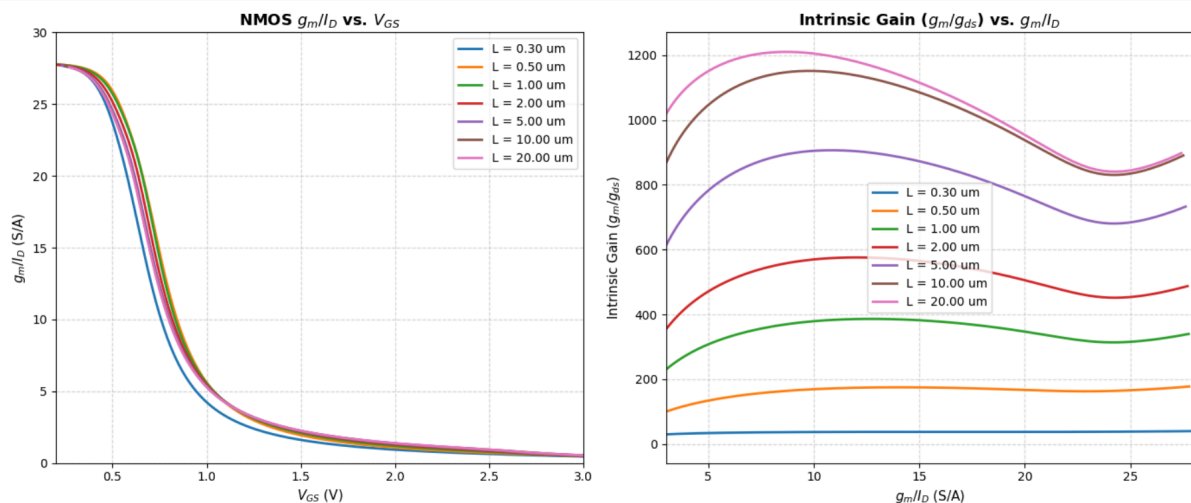
# Plot 2: gm/gds vs gm/ID
ax2.plot(gm_over_id[idx, :], intrinsic_gain[idx, :], label=label_text, linewidth=2)

# --- Format Plot 1 ---
ax1.set_title("NMOS  $g_m/I_D$  vs.  $V_{GS}$ ", fontsize=13, fontweight='bold')
ax1.set_xlabel(" $V_{GS}$  (V)", fontsize=11)
ax1.set_ylabel(" $g_m/I_D$  (S/A)", fontsize=11)
ax1.grid(True, which="both", linestyle="--", alpha=0.5)
ax1.set_xlim(0.2, 3.0) # Clipping early cutoff region for better focus
ax1.set_ylim(0, 30)
ax1.legend()

# --- Format Plot 2 ---
ax2.set_title("Intrinsic Gain ( $g_m/g_{ds}$ ) vs.  $g_m/I_D$ ", fontsize=13, fontweight='bold')
ax2.set_xlabel(" $g_m/I_D$  (S/A)", fontsize=11)
ax2.set_ylabel("Intrinsic Gain ( $g_m/g_{ds}$ )", fontsize=11)
ax2.grid(True, which="both", linestyle="--", alpha=0.5)
# Using a log scale for Gain is standard practice to see short vs long channel differences
# clearly
# ax2.set_yscale('log')
ax2.set_xlim(3, 28)
ax2.legend()

plt.tight_layout()
plt.show()

```



Saving LUTs

```

import os
import numpy as np

# Define save path

```

```

save_filename = "gf180_lut_data.npz"
save_path = os.path.join(w_dir, save_filename)

# Save the arrays
# We unpack the dictionary keys directly into np.savez using the ** double-asterisk trick
np.savez_compressed(
    save_path,
    L_vector=L_vector,
    VX_vector=VX_vector,
    **LUT_Matrices
)

print(f"Success! LUT datasets saved securely to:\n{save_path}")

```

LUT FINDER TOOL

```

import numpy as np
from scipy.interpolate import interp1d
import warnings

class LUT_Finder:
    def __init__(self, npz_filepath):
        """Initializes the Agent and pre-computes all methodology matrices."""
        data = np.load(npz_filepath)
        self.L_vec = data['L_vector']
        self.V_vec = data['VX_vector']
        self.W_ref = 10e-6 # 10um width used during LUT generation

        # Dictionary holding all N x M matrices for fast lookup
        self.db = {'nmos': {}, 'pmos': {}}
        self._build_matrices(data)

    def _build_matrices(self, data):
        """Constructs the derived parameter tensors for g_m/I_D methodology."""
        for mos, pfx in zip(['nmos', 'pmos'], ['@m1', '@m2']):
            # Raw SPICE vectors (PMOS currents and caps are taken as absolute)
            id_raw = np.abs(data[f'{pfx}[id]'])
            gm_raw = data[f'{pfx}[gm]']
            gds_raw = data[f'{pfx}[gds]']
            cgs = np.abs(data[f'{pfx}[cgs]'])
            cgd = np.abs(data[f'{pfx}[cgd]'])
            cgb = np.abs(data[f'{pfx}[cgb]'])

            # Mask out sub-picoamp currents to prevent division by zero in cutoff
            id_clean = np.where(id_raw > 1e-12, id_raw, np.nan)
            gds_clean = np.where(gds_raw > 1e-12, gds_raw, np.nan)

            # 1. Calculate Core Metrics

```

```

self.db[mos]['gm_ID'] = gm_raw / id_clean
self.db[mos]['gm_gds'] = gm_raw / gds_clean
self.db[mos]['gds_ID'] = gds_clean / id_clean
self.db[mos]['ID_W'] = id_clean / self.W_ref

# fT = gm / (2 * pi * Cgg) where Cgg is approx Cgs + Cgd + Cgb
cgg = cgs + cgd + cgb
self.db[mos]['fT'] = gm_raw / (2 * np.pi * cgg)

# VGS and Vth
self.db[mos]['VGS'] = self.V_vec # V_X maps to VGS or VSG directly
self.db[mos]['Vth'] = data[f'{pfx}{vth}']

# =====
# 2D Surface Search Functions (L is Unknown)
# =====

def find_L_given_fT_gmgds(self, fT_Q, gm_gds_Q, mos_type):
    """1. Finds L given f_T and gm/gds (Intrinsic Gain)."""
    fT_mat = self.db[mos_type]['fT']
    gmgds_mat = self.db[mos_type]['gm_gds']

    # Calculate normalized 2D error surface to find the closest intersection
    error = ((fT_mat - fT_Q) / fT_Q)**2 + ((gmgds_mat - gm_gds_Q) / gm_gds_Q)**2
    idx_L, _ = np.unravel_index(np.nanargmin(error), error.shape)
    return self.L_vec[idx_L]

def find_L_given_gdsID_VGS(self, gds_ID_Q, VGS_Q, mos_type):
    """6. Finds L given gds/ID (lambda) and VGS/VSG."""
    # Fixed: Using np.argmin and np.nanargmin functions properly
    idx_V = np.argmin(np.abs(self.V_vec - VGS_Q))
    gdsID_col = self.db[mos_type]['gds_ID'][:, idx_V]
    idx_L = np.nanargmin(np.abs(gdsID_col - gds_ID_Q))
    return self.L_vec[idx_L]

def find_L_given_gdsID_gmID(self, gds_ID_Q, gm_ID_Q, mos_type):
    """9. Finds L given gds/ID (lambda) and gm/ID."""
    gdsID_mat = self.db[mos_type]['gds_ID']
    gmID_mat = self.db[mos_type]['gm_ID']

    error = ((gdsID_mat - gds_ID_Q) / gds_ID_Q)**2 + ((gmID_mat - gm_ID_Q) / gm_ID_Q)**2
    idx_L, _ = np.unravel_index(np.nanargmin(error), error.shape)
    return self.L_vec[idx_L]

# =====
# 1D Vector Search Functions (L is Known)
# =====

def _get_L_row(self, L_Q, mos_type, param_name):

```

```

        """Helper to extract a 1D parameter array for a specific Length."""
        # Fixed: Using np.argmin
        idx_L = np.argmin(np.abs(self.L_vec - L_Q))
        return self.db[mos_type][param_name][idx_L, :]

def _interpolate_1d(self, x_array, y_array, x_target, param_name="Value"):
    """Helper to safely interpolate values, handling out-of-bounds cleanly."""
    valid_idx = ~np.isnan(x_array) & ~np.isnan(y_array)
    x_clean, y_clean = x_array[valid_idx], y_array[valid_idx]

    # 1. Sort to prevent non-monotonic crashes
    sort_idx = np.argsort(x_clean)
    x_clean, y_clean = x_clean[sort_idx], y_clean[sort_idx]

    # 2. Remove duplicate x-values (SPICE numerical noise can create duplicates)
    x_clean, unique_idx = np.unique(x_clean, return_index=True)
    y_clean = y_clean[unique_idx]

    # 3. Use bounds_error=False and fill_value=np.nan instead of extrapolating
    f = interp1d(x_clean, y_clean, kind='linear', bounds_error=False, fill_value=np.nan)

    result = float(f(x_target))

    # 4. Warn the user if they request a physically impossible operating point
    if np.isnan(result):
        min_val, max_val = np.nanmin(x_clean), np.nanmax(x_clean)
        warnings.warn(
            f"Target {x_target} is out of bounds for this L. "
            f"Valid range is [{min_val:.3e}, {max_val:.3e}]. Returning NaN."
        )

    return result

def find_ID_W_given_L_gmgds(self, L_Q, gm_gds_Q, mos_type):
    """2. Finds ID/W given L and gm/gds."""
    gmgds_row = self._get_L_row(L_Q, mos_type, 'gm_gds')
    idw_row = self._get_L_row(L_Q, mos_type, 'ID_W')
    return self._interpolate_1d(gmgds_row, idw_row, gm_gds_Q)

def find_gm_ID_given_L_gmgds(self, L_Q, gm_gds_Q, mos_type):
    """3. Finds gm/ID given L and gm/gds."""
    gmgds_row = self._get_L_row(L_Q, mos_type, 'gm_gds')
    gmID_row = self._get_L_row(L_Q, mos_type, 'gm_ID')
    return self._interpolate_1d(gmgds_row, gmID_row, gm_gds_Q)

def find_VGS_given_L_gmID(self, L_Q, gm_ID_Q, mos_type):
    """4. Finds VGS/VSG given L and gm/ID."""
    gmID_row = self._get_L_row(L_Q, mos_type, 'gm_ID')
    return self._interpolate_1d(gmID_row, self.V_vec, gm_ID_Q)

```

```

def find_Vth_given_L_gmID(self, L_Q, gm_ID_Q, mos_type):
    """5. Finds Vth given L and gm/ID."""
    gmID_row = self._get_L_row(L_Q, mos_type, 'gm_ID')
    vth_row = self._get_L_row(L_Q, mos_type, 'Vth')
    return self._interpolate_1d(gmID_row, vth_row, gm_ID_Q)

def find_gm_ID_given_L_VGS(self, L_Q, VGS_Q, mos_type):
    """7. Finds gm/ID given L and VGS/VSG."""
    gmID_row = self._get_L_row(L_Q, mos_type, 'gm_ID')
    return self._interpolate_1d(self.V_vec, gmID_row, VGS_Q)

def find_ID_W_given_L_gmID(self, L_Q, gm_ID_Q, mos_type):
    """8 & 10. Finds ID/W given L and gm/ID."""
    gmID_row = self._get_L_row(L_Q, mos_type, 'gm_ID')
    idw_row = self._get_L_row(L_Q, mos_type, 'ID_W')
    return self._interpolate_1d(gmID_row, idw_row, gm_ID_Q)

```

Verify Finder Tool

```

# 1. Initialize the agent tool
finder = LUT_Finder("gf180_lut_data.npz")

# 2. Get L from fT and gm/gds
L_1_2 = finder.find_L_given_fT_gmgds(fT_Q=1e9, gm_gds_Q=100, mos_type='nmos')

# 3. Get ID/W and gm/ID based on that Length
ID_W_1_2 = finder.find_ID_W_given_L_gmgds(L_Q=L_1_2, gm_gds_Q=100, mos_type='nmos')
gm_ID_1_2 = finder.find_gm_ID_given_L_gmgds(L_Q=L_1_2, gm_gds_Q=100, mos_type='nmos')

VGS_1_2=finder.find_VGS_given_L_gmID(L_Q=L_1_2, gm_ID_Q=gm_ID_1_2, mos_type='nmos')

Vth_1_2=finder.find_Vth_given_L_gmID(L_Q=L_1_2, gm_ID_Q=gm_ID_1_2, mos_type='nmos')

print(f"Diff Pair L: {L_1_2} | ID/W: {ID_W_1_2} | gm/ID: {gm_ID_1_2} | VGS : {VGS_1_2} | Vth : {Vth_1_2}")

```

```

Diff Pair L: 2.70000000000000025e-06 | ID/W: 44.07482941513996 | gm/ID: 1.1937390471638467 | VGS : 2.1226558430590114 | Vth : 0.663318328760571

```

```

import os
import numpy as np

# 1. Load your data
# Ensure w_dir matches where you saved gf180_lut_data.npz
w_dir = r"C:\Users\Mohamed\Desktop\ML\CAD_projects\Netlists\GF_180nm_tech_work\project"
data = np.load(os.path.join(w_dir, "gf180_lut_data.npz"))

```

```

L_vec = data['L_vector']
V_vec = data['VX_vector']

# 2. Define your test point (The L and VGS you want to check)
target_L = 2.7e-6 # 0.5 um
target_VGS = 2.122 # 1.2 V

# 3. Find the nearest index for L and VGS
# np.abs(...).argmin() finds the index of the value closest to your target
idx_L = (np.abs(L_vec - target_L)).argmin()
idx_V = (np.abs(V_vec - target_VGS)).argmin()

print(f"Index L: {idx_L} (Actual L: {L_vec[idx_L]*1e6:.2f} um)")
print(f"Index V: {idx_V} (Actual VGS: {V_vec[idx_V]:.2f} V)")

# 4. Extract data from the matrices
# We look at NMOS (@m1)
gm = data['@m1[gm]'][idx_L, idx_V]
id_val = np.abs(data['@m1[id]'][idx_L, idx_V])
gds = data['@m1[gds]'][idx_L, idx_V]
cgg = np.abs(data['@m1[cgs]'][idx_L, idx_V] + data['@m1[cgd]'][idx_L, idx_V] + data['@m1[cgb]']
[idx_L, idx_V])

# 5. Compute derived parameters
fT = gm / (2 * np.pi * cgg)
gm_ID = gm / id_val
gm_gds = gm / gds

print("-" * 30)
print(f"VERIFICATION RESULTS:")
print(f"fT      = {fT/1e9:.2f} GHz")
print(f"gm/ID   = {gm_ID:.2f} S/A")
print(f"gm/gds  = {gm_gds:.2f} (Intrinsic Gain)")

```

```

Index L: 240 (Actual L: 2.70 um)
Index V: 212 (Actual VGS: 2.12 V)
-----
VERIFICATION RESULTS:
fT      = 1.00 GHz
gm/ID   = 1.20 S/A
gm/gds  = 100.78 (Intrinsic Gain)

```